

Documentación Técnica: Sistema de Estadísticas e Inventario

Este documento detalla la arquitectura de los sistemas de estadísticas (Stats) y la gestión de ítems en Visceral Limbo. Estos sistemas están altamente acoplados, ya que los ítems frecuentemente actúan como modificadores de las estadísticas base de los personajes a través del inventario.

1. Sistema de Estadísticas (Stat System)

El sistema de estadísticas permite definir, modificar y consultar valores numéricos de forma escalable. Utiliza ScriptableObjects para evitar "Hardcoding" y facilitar el diseño desde el editor de Unity.

StatsManager

Es el componente principal adosado a entidades (como el jugador o los enemigos). Gestiona un diccionario que relaciona un identificador único con su respectiva estadística.

- **Inicialización:** Carga un `StatBlockData` (ScriptableObject) que contiene las estadísticas base. Clona profundamente estos valores (usando `new FloatStat(stat)`) para no sobrescribir los valores originales del ScriptableObject.
- **Consulta:** `GetFloatStatValue(StatIdentifier StatID)` devuelve el valor final calculado.
- **Modificación:** `UpdateFloatStatValue(...)` recibe un modificador y su ID para aplicarlo, disparando el evento `OnStatChanged`.
- **Eliminación:** `RemoveFloatStatModifier(...)` permite retirar modificadores específicos, útil para buffs temporales o desequipado de ítems.

Stat y FloatStat

La clase abstracta `Stat` define la estructura base. `FloatStat` es su implementación concreta para números de coma flotante.

RecalculateStat(): Esta función es vital. Calcula el valor final aplicando todos los modificadores activos en orden:

1. Se ordenan los modificadores por su **ModifierType** : primero **flat** , luego **percentAdd** , y finalmente **PercentMult** .
2. **Flat**: Suma directa (ej. +10 de daño).
3. **PercentAdd**: Suma porcentual basada en el valor base (ej. +20% del daño base).
4. **PercentMult**: Multiplicación final que escala sobre los resultados anteriores (ej. x1.5 daño total).

Identificadores de Stats

Para evitar strings frágiles, las estadísticas se buscan mediante **StatIdentifier** , un ScriptableObject vacío que actúa como una llave única (Key) en los diccionarios.

2. Sistema de Inventario y Gestión de Ítems

El inventario rastrea los ítems recolectados, maneja el "Stacking" (acumulación del mismo ítem) y dispara sus efectos en distintos contextos (por frame o al golpear).

InventoryManager

Componente que centraliza la lógica de recolección.

- **Diccionario de Ítems:** Usa **Dictionary<ItemDefinitionSO, ItemLogic>** para mapear la definición del ítem con su instancia lógica en el juego.
- **AddItemStack():** Verifica si el ítem existe. Si es nuevo, lo instancia y lo clasifica (¿Es un ítem activo? ¿Se dispara al golpear?). Si ya existe, incrementa su contador de stacks (**AddStack()**).
- **Ciclo de Update:** Recorre la lista **_ActiveItems** e invoca **UpdateActiveItem()** de la interfaz **I_ItemActiveItem** , permitiendo a los ítems tener lógica continua.
- **Eventos de Daño:** **ProcOnHitEffects()** es llamado por el sistema de combate, iterando sobre los ítems de tipo **I_OnHitItem** para aplicar efectos adicionales (ej. sangrado, daño crítico).

ItemDefinitionSO

Un ScriptableObject que define la "metadata" del ítem: ID, Nombre, Descripción, Ícono (Sprite) y los valores de buffs por stack. Contiene la referencia al Prefab que contiene la lógica real del ítem (**ItemLogic**).

Jerarquía de Ítems e Interfaces

La lógica individual de cada ítem reside en clases que heredan de `ItemLogic` y aplican diversas interfaces según su función:

- `ItemLogic` : Maneja la recolección física (`OnTriggerEnter`), el registro en el inventario y el conteo de Stacks.
- `I_ItemActiveItem` : Ítems que requieren lógica por frame.
- `I_ItemPassiveItem` / `I_ItemStatModifier` : Ítems diseñados para modificar directamente al `StatsManager` .

3. Guía de Uso Rápido

Para implementar rápidamente una nueva estadística o ítem y conectarlo con tu sistema o UI, sigue estos pasos:

1. **Creación de Datos:** Crea un nuevo `StatIdentifier` (ej. `ID_Health`) y agrégalo a tu `StatBlockData` base desde el inspector de Unity.
2. **Suscripción a Eventos (Evitar Polling):** Para actualizar la UI o disparar VFX, **evita hacer query** con `GetFloatStatValue()` dentro de un `Update()` . En su lugar, suscríbete al Action `OnStatChanged` del `StatsManager` .
Ejemplo: `_StatsManager.OnStatChanged += MiFuncionUI;`
3. **Aplicar Modificadores:** Cuando recojas un ítem que afecte estadísticas, crea una instancia de tu modificador y llama a `UpdateFloatStatValue(ID_Health, NuevoModificador)` . Esto automáticamente recalculará el valor final y disparará `OnStatChanged` con el valor actualizado para todos los suscriptores de manera eficiente.

4. Extensiones y Buenas Prácticas

Para asegurar que tu arquitectura se mantenga escalable y respete los principios SOLID, ten en cuenta las siguientes prácticas:

- **Principio Abierto/Cerrado (Open/Closed):** Si necesitas que un ítem tenga un comportamiento completamente nuevo (por ejemplo, que estalle al matar a un enemigo), no modifiques la clase `InventoryManager` . Usa interfaces que ya tienes (como `I_ItemOnKill`) y haz que el script del ítem la implemente. Tu sistema que gestiona las muertes solo debe buscar si los ítems tienen esa interfaz e invocarla.
- **Inmutabilidad de ScriptableObjects:** Asegúrate de que tus SOs (`ItemDefinitionSO` , `StatBlockData`) se mantengan como contenedores de datos inmutables en tiempo de ejecución. El patrón de clonar con `new FloatStat(stat)` en el Awake del `StatsManager` es una excelente

práctica; mantenlo así para evitar que los valores de diseño se corrompan permanentemente en el Editor durante el Play Mode.

- **Gestión de Memoria (Unsubscribe):** Siempre que un script (como una barra de vida de un enemigo o del HUD del jugador) se suscriba a eventos como `OnStatChanged` o el `ItemPickUp` del inventario, **debes** asegurar de desuscribirte en los métodos `OnDisable()` u `OnDestroy()` (ej. `_StatsManager.OnStatChanged -= MiFuncionUI;`). De lo contrario, generarás Memory Leaks y NullReferenceExceptions.
- **Responsabilidad Única (SRP) con Identificadores:** Mantén el uso del `EffectID` limpio en los `StatModifiers`. Usa IDs predecibles (por ejemplo, el ID o nombre único del ítem) para que métodos como `RemoveFloatStatModifier` funcionen de manera robusta sin depender de lógicas externas acopladas.